

Tutorial 2: Raspberry Pi LED and Flex Sensor

This tutorial introduces the Raspberry Pi GPIO system through a simple LED circuit and then extends the same breadboard setup with a **flex sensor** and an **MCP3008 analog-to-digital converter (ADC)**.

By the end of this tutorial, you will be able to:

- identify the main Raspberry Pi, breadboard, LED, flex sensor, and ADC components
- connect and test an LED using a Raspberry Pi GPIO pin
- understand why an ADC is needed for an analog flex sensor
- read the changing flex-sensor value through the MCP3008
- use the amount of bending to control LED brightness gradually
- connect the experiment to the idea of linguistic variables and fuzzy reasoning

Important: Run the notebook section by section. Do not use **Run All**, because the final section contains an optional Raspberry Pi shutdown command.

0. Raspberry Pi Initial Setup

Before beginning this tutorial, the Raspberry Pi Zero 2W must be prepared and connected to your computer.

This setup is normally required only once. After completing it, the Raspberry Pi can be reused for the following tutorials.

What you need

- Raspberry Pi Zero 2 W
- microSD card
- microSD card reader/adaptor
- micro-USB power supply

Recommended Network Setup

For these tutorials, using a **mobile hotspot** is recommended.

Connect both the Raspberry Pi and your computer to the same hotspot. This provides a simple and portable network connection and avoids problems with Wi-Fi networks that require browser-based login.

Note: Use your own hotspot name and password. Keep them unchanged after configuring the Raspberry Pi so that it can reconnect automatically.

0.1 Prepare the microSD Card

The Raspberry Pi boots from a microSD card containing Raspberry Pi OS.

To prepare the card, download **Raspberry Pi Imager** from the official Raspberry Pi website:

[Download Raspberry Pi Imager](#)

Choose your operating system below.

► macOS

1. Open the Raspberry Pi Imager download page.
2. Download **Raspberry Pi Imager for macOS**.
3. Install the application.
4. Insert the microSD card into a microSD card reader/adapter.
5. Connect the reader to your Mac.
6. Open **Raspberry Pi Imager**.

► Windows

1. Open the Raspberry Pi Imager download page.
2. Download **Raspberry Pi Imager for Windows**.
3. Run the installer and complete the installation.
4. Insert the microSD card into a card reader/adapter.
5. Connect the reader to your computer.
6. Open **Raspberry Pi Imager**.

► Linux

1. Open the Raspberry Pi Imager download page.
2. Follow the installation option provided for your Linux system.
3. Insert the microSD card into a card reader/adapter.
4. Connect the reader to your computer.
5. Open **Raspberry Pi Imager**.

Once Raspberry Pi Imager is open, the remaining setup steps are the same for all operating systems.

0.2 Select the Device, Operating System, and Storage

Raspberry Pi Imager guides you through the setup in the following order:

Device → Operating System → Storage → Customisation → Writing

1. Device

Select:

Raspberry Pi Zero 2 W

Make sure you select **Raspberry Pi Zero 2 W**, not Raspberry Pi Zero or another Raspberry Pi model.

Setup steps

- Device
- OS
- Storage
- Customisation
- Writing
- Done

Select your Raspberry Pi device

- Raspberry Pi 5**
Raspberry Pi 5, 500 / 500+, and Compute Module 5
- Raspberry Pi 4**
Raspberry Pi 4 Model B, 400, and Compute Module 4 / 4S
- Raspberry Pi 3**
Raspberry Pi 3 Model A+ / B / B+ and Compute Module 3 / 3+
- Raspberry Pi 2**
Raspberry Pi 2 Model B
- Raspberry Pi 1**
Raspberry Pi 1 Model A / A+ / B / B+ and Compute Module 1
- Raspberry Pi Zero 2 W**
Raspberry Pi Zero 2 W
- Raspberry Pi Zero**
Raspberry Pi Zero, Zero W, and Zero WH

No filtering
Show every possible image

APP OPTIONS

NEXT

2. Operating System

Select:

Raspberry Pi OS (64-bit)

Choose the standard **Raspberry Pi OS (64-bit)** option with the Raspberry Pi Desktop.

Do not select:

- Raspberry Pi OS (32-bit)
- Raspberry Pi OS (Legacy, 32-bit)
- Raspberry Pi OS Lite
- another general-purpose operating system

3. Storage

Select the microSD card connected to your computer.

Check the displayed storage size carefully to make sure that you are selecting the microSD card and not another storage device.

Warning: Writing Raspberry Pi OS will erase everything currently stored on the selected microSD card.

After selecting the device, operating system, and storage, continue to **Customisation**.

0.3 Configure Raspberry Pi OS

Before Raspberry Pi OS is written to the microSD card, configure the Raspberry Pi using the **Customisation** steps in Raspberry Pi Imager.

Complete the following settings.

Hostname

Enter:

```
cwpi-zero2w
```

The hostname is the name used to identify the Raspberry Pi on the network.

Later, it will allow you to connect using:

```
cwpi-zero2w.local
```

Localisation

Choose the settings appropriate for your location.

When working in Switzerland, use:

- **Time zone:** Europe/Zurich
- **Wi-Fi country:** CH
- **Keyboard layout:** select the layout used by your computer

User

Create the Raspberry Pi user account.

Example username:

```
pi
```

Create your own password and keep it securely.

You may use another username. If you do, replace `pi` with your username in the SSH commands shown later in this tutorial.

Wi-Fi

Enter the details of the mobile hotspot that will be used with the Raspberry Pi:

- **SSID:** your mobile hotspot name
- **Password:** your mobile hotspot password

Enter the SSID and password exactly as they appear on your phone.

For these tutorials, a mobile hotspot is recommended because both the computer and Raspberry Pi can connect to the same portable network without depending on university or public Wi-Fi.

Keep the hotspot name and password unchanged after this setup. The Raspberry Pi will use these saved details to reconnect automatically.

Remote Access

Enable:

SSH

Select:

Use password authentication

SSH allows the Raspberry Pi to be controlled remotely from the terminal on your computer.

Raspberry Pi Connect

Leave **Raspberry Pi Connect** disabled.

It is not required for these tutorials because SSH and JupyterLab will be used to access the Raspberry Pi.

Important: No SSH file or configuration file needs to be manually copied onto the microSD card. Raspberry Pi Imager prepares the user account, network configuration, and SSH access during this setup.

0.4 Connect to the Raspberry Pi Using SSH

SSH allows you to control the Raspberry Pi from the terminal on your computer.

Both the Raspberry Pi and your computer must be connected to the same mobile hotspot.

► macOS

Open **Terminal** and enter:

```
ssh pi@cwpi-zero2w.local
```

► Windows

Open **Windows Terminal** or **PowerShell** and enter:

```
ssh pi@cwpi-zero2w.local
```

► Linux

Open a terminal and enter:

```
ssh pi@cwpi-zero2w.local
```

During the first connection, you may see:

```
Are you sure you want to continue connecting  
(yes/no/[fingerprint])?
```

Enter:

```
yes
```

Then enter the Raspberry Pi password created during the Raspberry Pi Imager setup.

A successful connection should display something similar to:

```
pi@cwpi-zero2w:~ $
```

You are now connected to the Raspberry Pi.

Note: The commands shown below are terminal commands. Copy or type them into the terminal connected to the Raspberry Pi. They are not intended to be run directly as Jupyter Notebook code cells.

If SSH does not connect on a later session

Occasionally the Pi may take longer to start or may not be reachable at its usual address. First check that its power supply is connected, wait a minute for startup, and confirm that your computer and the Pi are on the **same hotspot/network**. If the hotspot name or password changed, the Pi may not join it. Retry `ssh pi@cwpi-zero2w.local`; if the hostname is not found, check the hotspot's connected devices for the Pi's IP address and try `ssh pi@<Pi-IP-address>`.

If the connection still fails, allow the Pi to cool if it feels unusually hot, then try again later or on the next day. Heat is **one possible issue**, not a diagnosis from an SSH error. Avoid repeatedly unplugging the Pi while it is running. When you finish a session, use the safe shutdown procedure in **Section 17** and wait for shutdown before removing power. This reduces the risk of microSD card corruption and can make the next startup more reliable.

0.5 Prepare Raspberry Pi OS

After establishing the SSH connection, update the Raspberry Pi.

Enter the following command in the terminal:

```
sudo apt-get update
```

Then install the available updates:

```
sudo apt-get upgrade -y
```

Install the basic tools required for the tutorials:

```
sudo apt-get install -y git python3-pip python3-venv python3-gpiozero
```

If the Raspberry Pi was updated, reboot it:

```
sudo reboot
```

The SSH connection will close automatically.

Wait approximately 2 minute and reconnect:

```
ssh pi@cwpi-zero2w.local
```

When you again see a prompt similar to:

```
pi@cwpi-zero2w:~ $
```

the Raspberry Pi is ready for the next setup step.

0.6 Set Up JupyterLab

The tutorials are provided as Jupyter Notebooks.

JupyterLab runs on the Raspberry Pi, while the notebook interface is viewed in a web browser on your computer.

This setup is normally required only once.

From the SSH terminal, create the course Python environment:

```
python3 -m venv --system-site-packages ~/cw_jupyter
```

Activate the environment:

```
source ~/cw_jupyter/bin/activate
```

The terminal prompt should now begin with something similar to:

```
(cw_jupyter)
```

Install JupyterLab and the Jupyter kernel:

```
python -m pip install --upgrade pip
python -m pip install jupyterlab ipykernel
```

Register the course environment as a Jupyter kernel:

```
python -m ipykernel install --user --name cw-jupyter --display-name
"CW Jupyter"
```

The Jupyter environment is now prepared.

0.7 Start and Open JupyterLab

Whenever you want to work with the course tutorials, first connect to the Raspberry Pi through SSH:

```
ssh pi@cwpi-zero2w.local
```

Activate the course environment:

```
source ~/cw_jupyter/bin/activate
```

Start JupyterLab:

```
jupyter lab --no-browser --port=8888
```

Keep this terminal window open.

JupyterLab will display information including an access token.

Now open a **second terminal window** on your computer.

Enter:

```
ssh -L 8890:localhost:8888 pi@cwpi-zero2w.local
```

Enter your Raspberry Pi password when requested.

Keep this second terminal open as well.

Now open a web browser and enter:

```
http://localhost:8890
```

JupyterLab should open in the browser.

If a token is requested, copy the token displayed in the first terminal window.

The Jupyter Notebook is displayed on your computer, but the notebook code is executed on the Raspberry Pi. This allows the notebook to communicate with hardware connected to the Raspberry Pi.

Once JupyterLab is open successfully, continue with Section 0.8 below to open and run the provided tutorial notebook.

0.8 Open and Run This Tutorial Notebook

The course tutorials are provided as ready-to-use Jupyter Notebook (`.ipynb`) files.

After JupyterLab opens in your browser:

1. Place or download this tutorial folder onto the Raspberry Pi.
2. In JupyterLab, navigate to the tutorial folder.
3. Open **Tutorial 2 - Raspberry Pi LED and Flex Sensor.ipynb**.
4. Work through the notebook from top to bottom.

5. Run each code cell directly inside this notebook when instructed.

You do **not** need to create a new Jupyter Notebook or copy the code into another file.

The Markdown cells provide the setup, wiring, and experiment instructions, while the code cells contain the Python commands that run directly on the Raspberry Pi.

For later Raspberry Pi tutorials, you can reuse the same Raspberry Pi setup and the same `cw_jupyter` environment. Normally, you only need to connect through SSH, activate the environment, start JupyterLab, and open the next tutorial notebook.

Connection note: Occasionally, the Raspberry Pi may not reconnect immediately over SSH, especially after repeated use, long sessions, or changes in the network connection. If the Pi does not respond, first check that both the Raspberry Pi and your computer are connected to the same network or hotspot. If the problem continues, power the Pi down safely and try again after some time.

Long continuous use can also make the Raspberry Pi warm. If the board becomes unusually hot, allow it to cool before continuing.

Whenever possible, avoid disconnecting the Raspberry Pi directly from power. Use a safe shutdown instead. A safe shutdown reduces the risk of file-system problems and can also help the Pi start cleanly the next time you connect.

A safe-shutdown example is provided at the end of this tutorial.

1. Components Required

You will use:

- Raspberry Pi Zero 2 W
- breadboard
- LED
- jumper wires
- 220 Ω to 330 Ω resistor for the LED
- short flex sensor
- MCP3008 8-channel 10-bit ADC
- 47 k Ω resistor for the flex-sensor voltage divider
- microSD card and Raspberry Pi power supply
- a computer connected to the Raspberry Pi through SSH/JupyterLab

2. Component Overview

2.1 Raspberry Pi

The Raspberry Pi is the main computing device. It runs Python and controls the GPIO pins used to communicate with the external circuit.

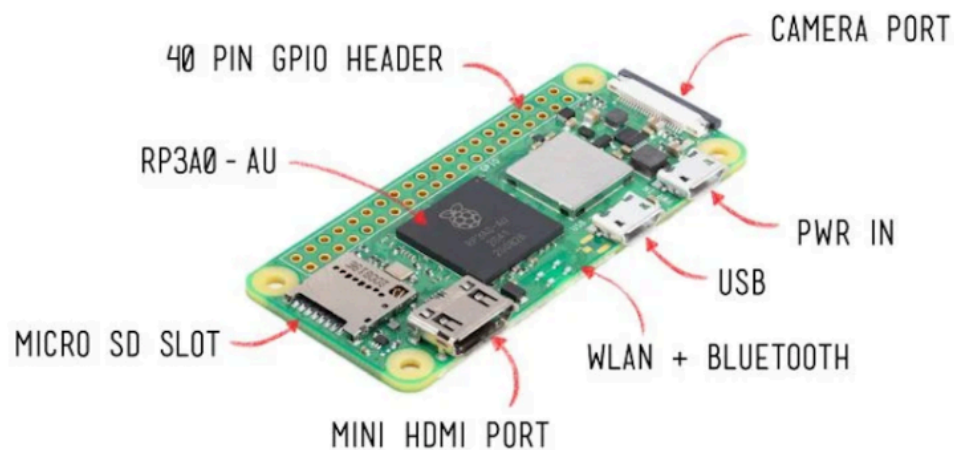


Figure 1: Raspberry Pi

2.2 Breadboard

A breadboard lets you build and modify circuits without soldering.

The middle section is divided by a central gap:

- holes **A–E** in one numbered row are connected
- holes **F–J** in the same numbered row are connected
- there is **no connection across the E–F centre gap**

This makes it possible to place components and integrated circuits without shorting their pins together.

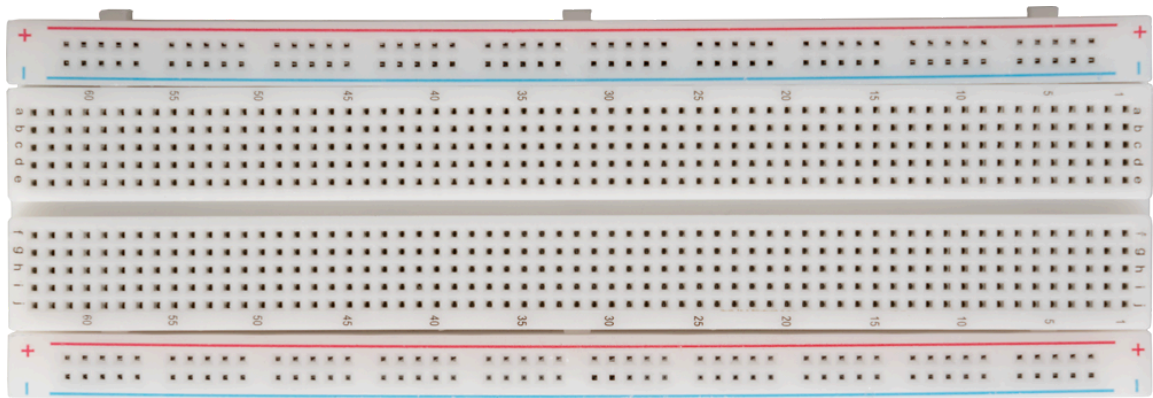


Figure 2: Breadboard

2.3 LED

The LED is the output device.

- long leg → **anode (+)**
- short leg → **cathode (-)**

A resistor must be placed in series with the LED to limit current.

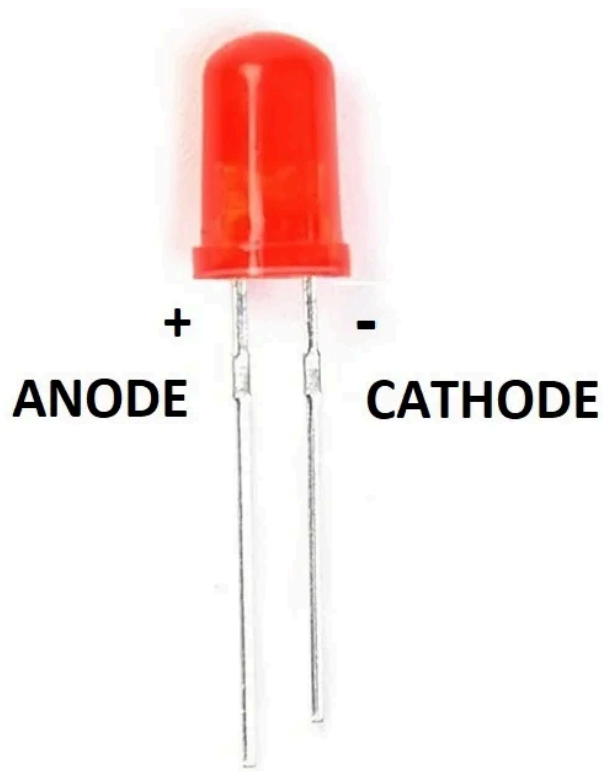


Figure 3: LED

2.4 Jumper Wires

Jumper wires make temporary connections between the Raspberry Pi and the breadboard. Male-to-female wires are especially useful for connecting the Pi GPIO header to the breadboard.

In Figure below: (Top to bottom) - Female-to-female, Male-to-male and Male-to-female respectively.



Figure 4: Jumper wires

2.5 Resistors

Two resistors have different jobs in this tutorial:

- **220–330 Ω :** limits current through the LED
- **47 k Ω :** forms a voltage divider with the flex sensor so its resistance change becomes a measurable voltage



Figure 5: Resistor

2.6 Flex Sensor

A flex sensor behaves like a variable resistor. Its resistance changes when the flexible sensing area is bent.

Imp Note: Hold the sensor close to its base and bend only the flexible upper section. Avoid repeatedly bending the delicate contact area where the two terminals join the sensor.

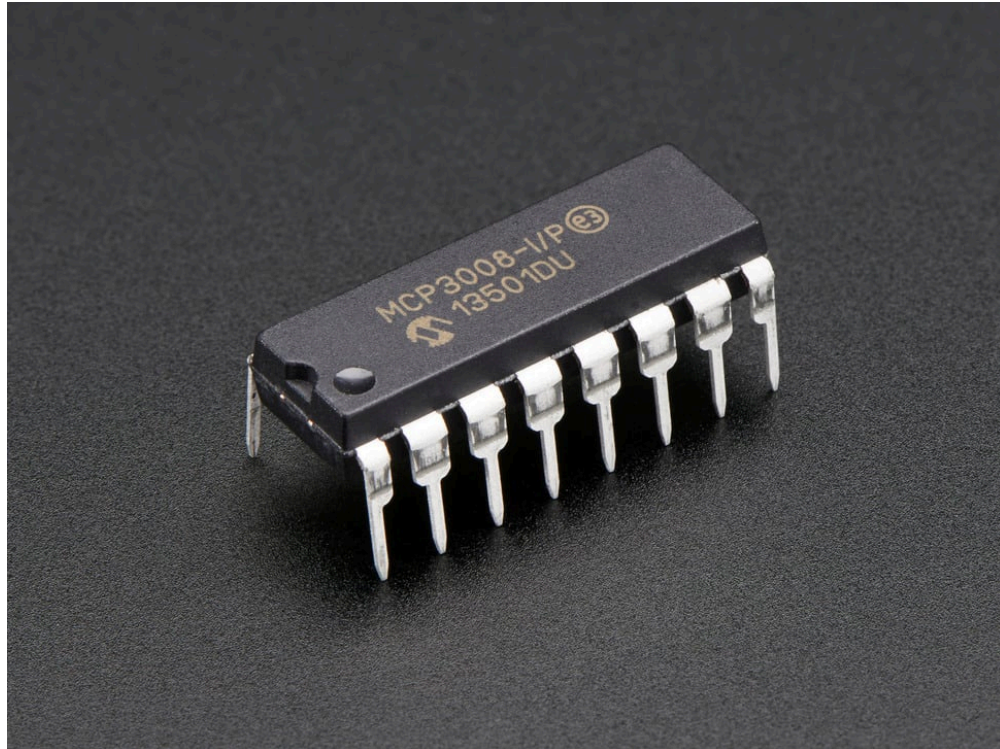


Figure 6: Flex sensor

2.7 MCP3008 ADC

The Raspberry Pi GPIO pins are digital and do not directly measure an analog voltage. The MCP3008 converts the changing analog voltage from the flex-sensor circuit into a digital value that Python can read.

The MCP3008 communicates with the Raspberry Pi using **SPI (Serial Peripheral Interface)**.

**Figure 7:** MCP3008 ADC

3. Safety Notes

Before wiring:

- disconnect power from the Raspberry Pi if you are unsure about a connection
- never connect the LED directly without its current-limiting resistor
- use **3.3 V**, not 5 V, for the analog side connected to the MCP3008
- check the MCP3008 orientation carefully before wiring its pins
- do not sharply fold the flex sensor at its contact/base area
- double-check GPIO, 3.3 V, and GND connections before running code
- keep the wiring neat so that accidental short circuits are easier to avoid

4. Raspberry Pi GPIO Reference

For this tutorial, the Raspberry Pi Zero 2 W is used.

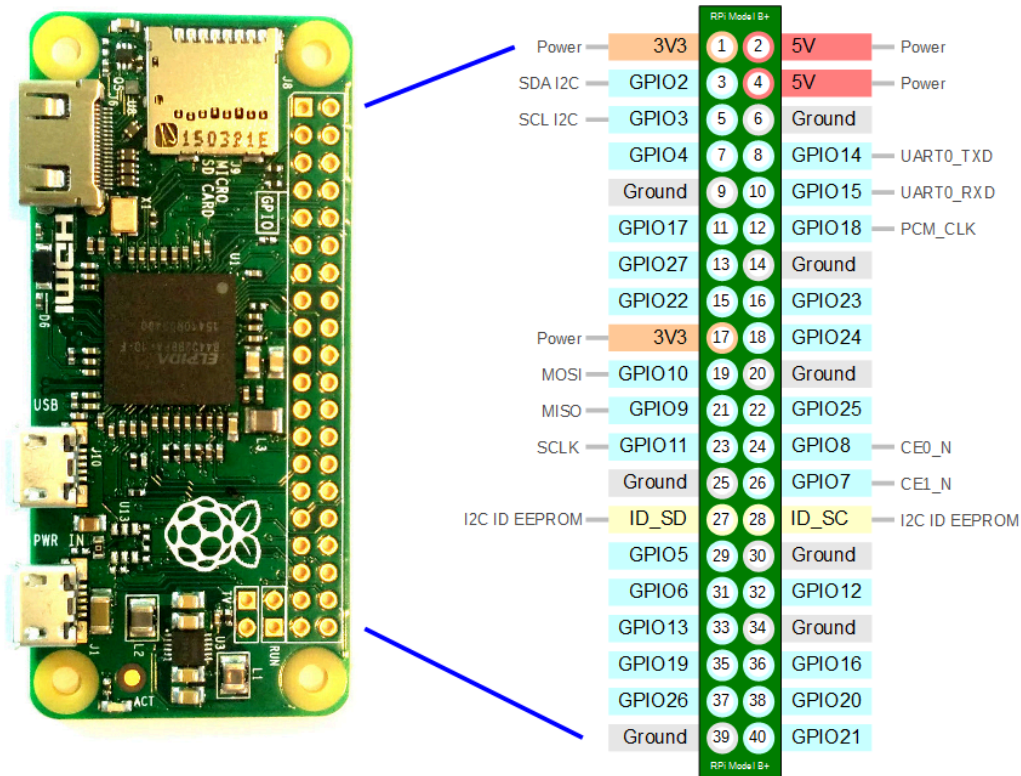


Figure 8: Raspberry Pi GPIO pinout

Important connections used in this tutorial include:

- **GPIO17 (physical pin 11)** → LED output
- **GND (physical pin 6)** → circuit ground
- **3.3 V** → MCP3008/flex-sensor analog circuit
- **SPI pins** → communication between the Raspberry Pi and MCP3008

The LED part of the circuit continues to use the same GPIO17 connection as the original experiment.

5. Enable SPI for the MCP3008

The MCP3008 communicates with the Raspberry Pi through SPI. This is a one-time Raspberry Pi configuration step.

Open a terminal connected to the Raspberry Pi and run:

```
sudo raspi-config
```

Then select:

Interface Options → SPI → Enable

If the Raspberry Pi asks for a reboot, reboot it once and reconnect through SSH/JupyterLab.

This does **not** reinstall Raspberry Pi OS, change the SD card setup, or alter the existing LED circuit. It simply enables the SPI interface already available on the Raspberry Pi.

6. Part A — LED Circuit

First, verify the original LED circuit before adding the flex-sensor input.

A simple LED circuit uses:

GPIO17 → LED → 220–330 Ω resistor → GND

For the existing example:

- LED long leg is connected toward **GPIO17 (physical pin 11)**
- LED short leg is connected through the resistor
- the other side of the resistor is connected to **GND**

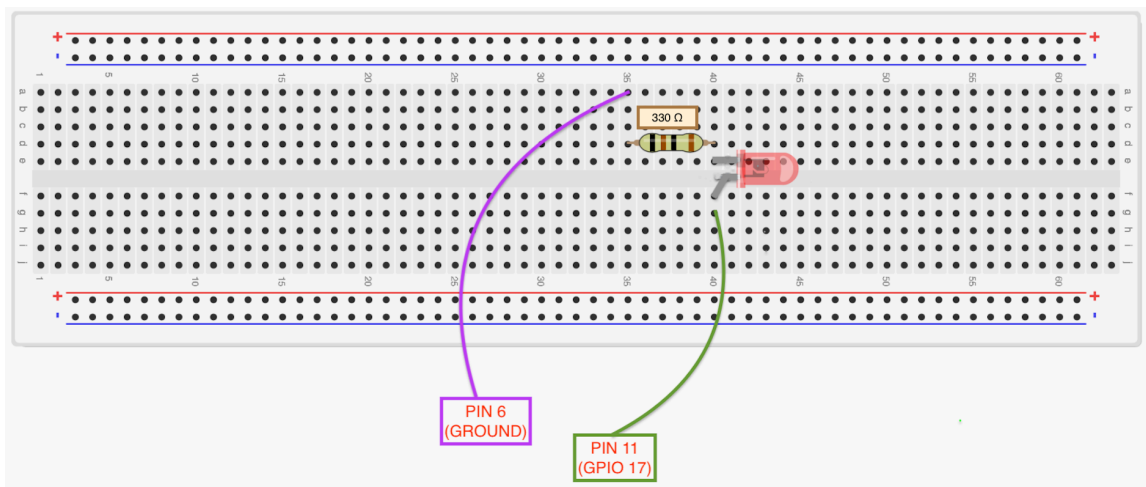


Figure 9: Existing LED circuit

7. Initial LED Test

Run this cell only after the LED circuit has been checked.

The LED should turn on for two seconds and then turn off.

```
In [ ]: from gpiozero import LED
        from time import sleep

        led = LED(17)

        print("Turning LED on")
        led.on()
        sleep(2)

        print("Turning LED off")
```

```
led.off()

led.close()
print("LED test completed")
```

8. LED Blink Test

This test confirms that the GPIO output responds repeatedly and reliably.

```
In [ ]: from gpiozero import LED
        from time import sleep

        led = LED(17)

        for i in range(5):
            print(f"Blink {i + 1}")
            led.on()
            sleep(1)
            led.off()
            sleep(1)

        led.close()
        print("Blink test completed")
```

9. Part B — Add the Flex Sensor and MCP3008

In Part A, the Raspberry Pi controlled an LED using a **digital GPIO output**.

In this part, we add a **flex sensor** so that the Raspberry Pi can react to how much the sensor is bent.

The flex sensor behaves like a variable resistor: its resistance changes when it bends. However, the Raspberry Pi GPIO pins cannot directly measure this continuously changing analog value. For this reason, we use an **MCP3008 Analog-to-Digital Converter (ADC)**.

The signal path is:

Flex sensor → voltage divider → MCP3008 → Raspberry Pi → LED

The flex sensor and a fixed resistor first create a voltage that changes with bending. The MCP3008 measures this voltage and sends the reading to the Raspberry Pi through SPI.

9.1 MCP3008 pinout

The MCP3008 has 16 pins. In this tutorial, only one analog input channel, **CH0**, is required.

When the notch of the MCP3008 is facing upward, pin 1 is at the upper-left side.

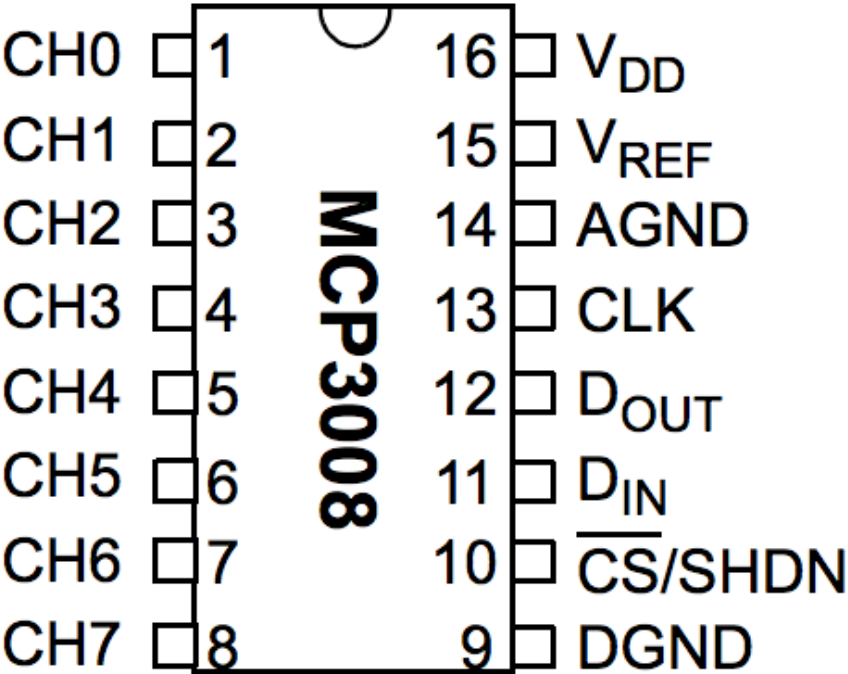


Figure 10: MCP3008 pinout

The pins used in this experiment are:

MCP3008 pin	Function	Raspberry Pi connection
1	CH0	Flex-sensor measurement point
9	DGND	GND
10	CS/SHDN	GPIO8 / CE0 — physical pin 24
11	DIN	GPIO10 / MOSI — physical pin 19
12	DOUT	GPIO9 / MISO — physical pin 21
13	CLK	GPIO11 / SCLK — physical pin 23
14	AGND	GND
15	VREF	3.3 V
16	VDD	3.3 V

Important: Use **3.3 V**, not 5 V, for the MCP3008 in this setup.

9.2 A note about the flex sensor

The flex sensor is delicate near its terminal/base area.

Keep the base as straight and stable as possible and bend only the flexible sensing section. Avoid sharply folding or twisting the sensor.

The two terminals of the flex sensor do not have a positive or negative polarity for this experiment.

9.3 How the flex sensor is connected

The flex sensor and a fixed resistor form a **voltage divider**.

The basic connection is:

3.3 V → Flex Sensor → Measurement Point → Fixed Resistor → GND

The **measurement point** is also connected to **CH0 (pin 1)** of the MCP3008.

Therefore, three connections meet at the measurement point:

- one terminal of the flex sensor,
- one side of the fixed resistor,
- MCP3008 CH0.

When the flex sensor bends, its resistance changes. This changes the voltage at the measurement point, which is then measured by the MCP3008.

9.4 Example breadboard connection used in this tutorial

The following example shows the breadboard arrangement used and tested for this tutorial.

MCP3008 placement

Place the MCP3008 across the centre gap of the breadboard, starting at row 27.

With the notch facing toward the lower row numbers:

MCP3008 pin	Breadboard position
1 — CH0	F27
2	F28
3	F29
4	F30
5	F31
6	F32
7	F33
8	F34

MCP3008 pin	Breadboard position
9 — DGND	E34
10 — CS/SHDN	E33
11 — DIN	E32
12 — DOUT	E31
13 — CLK	E30
14 — AGND	E29
15 — VREF	E28
16 — VDD	E27

MCP3008 to Raspberry Pi

Connect:

- **E27 / pin 16 (VDD)** → Raspberry Pi **3.3 V, physical pin 1**
- **E28 / pin 15 (VREF)** → Raspberry Pi **3.3 V, physical pin 17**
- **E29 / pin 14 (AGND)** → Raspberry Pi **GND, physical pin 9**
- **E30 / pin 13 (CLK)** → Raspberry Pi **GPIO11 / physical pin 23**
- **E31 / pin 12 (DOUT)** → Raspberry Pi **GPIO9 / physical pin 21**
- **E32 / pin 11 (DIN)** → Raspberry Pi **GPIO10 / physical pin 19**
- **E33 / pin 10 (CS/SHDN)** → Raspberry Pi **GPIO8 / physical pin 24**
- **E34 / pin 9 (DGND)** → Raspberry Pi **GND, physical pin 14**

Flex sensor and resistor

The 3.3 V connection from row 27 is shared with the flex-sensor circuit:

- Connect **A27 → A40** with a jumper wire.
- Place one flex-sensor terminal at **E40**.
- Place the other flex-sensor terminal at **E41**.
- Place one leg of the fixed resistor at **B41**.
- Place the other resistor leg at **B49**.
- Connect **A49 → Raspberry Pi GND, physical pin 39**.
- Connect **A41 → F27**, which is MCP3008 **CH0**.

Because holes A–E in the same numbered breadboard row are electrically connected:

- row 40 carries **3.3 V**,
- row 41 becomes the **measurement point**,
- row 49 is connected to **GND**.

The resulting path is:

3.3 V → Flex Sensor → Row 41 → Fixed Resistor → GND

and:

Row 41 → MCP3008 CH0

9.5 Keep the LED circuit connected

Keep the LED circuit from Part A in place.

The tested LED connection is:

GPIO17 (physical pin 11) → LED long leg → LED short leg → 330 Ω resistor → GND

In the example breadboard layout:

- LED short leg → **E21**
- LED long leg → **F21**

The LED, MCP3008, and flex-sensor circuit all share the Raspberry Pi ground.

Before powering the Raspberry Pi, check each connection one at a time.

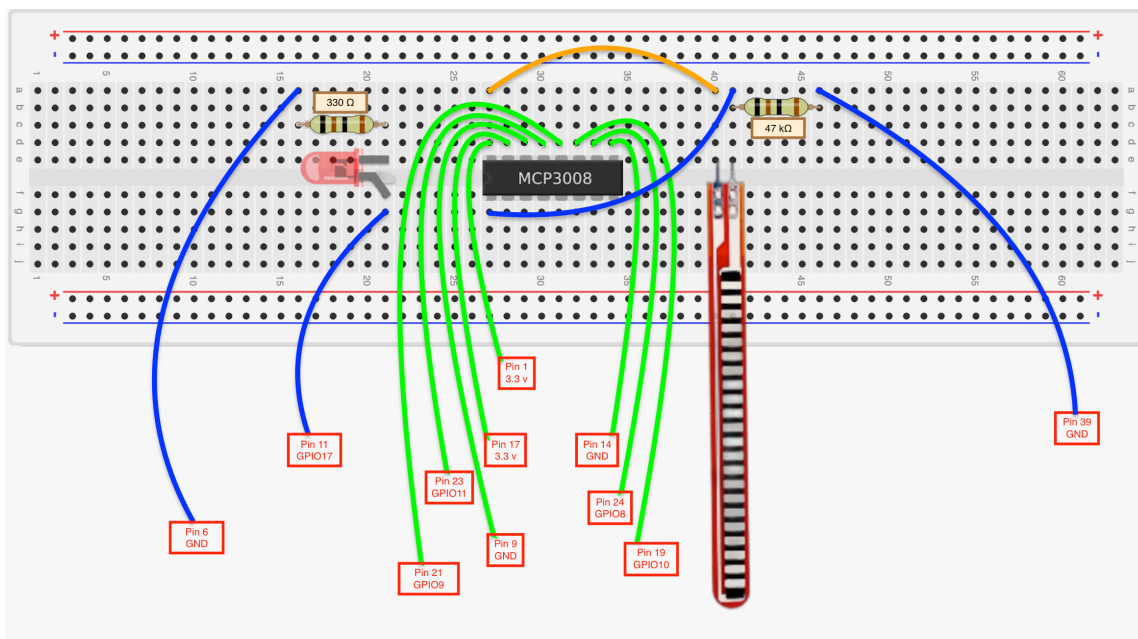


Figure 11: Raspberry Pi, MCP3008, flex sensor and LED wiring

The diagram above shows the complete example wiring used in this tutorial. The next step is to check whether the Raspberry Pi can read changing values from the flex sensor.

10. Check the Flex-Sensor Reading

The next cell reads MCP3008 channel 0.

Keep the sensor straight first and observe the value. Then bend only the flexible sensing area and observe how the value changes.

The normalized reading is between **0.0 and 1.0**.

Flex sensor bending tip:

For more consistent readings, keep the branded side on the inside of the bend and the sensing-strip side on the outside. Hold the sensor near its base and bend only the flexible sensing section. Avoid sharply folding or twisting the sensor, especially near the terminals.

```
In [ ]: from gpiozero import MCP3008
        from time import sleep

        flex = MCP3008(channel=0)

        print("Flex-sensor readings:")
        for _ in range(20):
            print(f"{flex.value:.3f}")
            sleep(0.25)

        flex.close()
```

11. First Flex-Sensor Test — Bend = LED ON

This first test keeps the behaviour intentionally simple.

1. Keep the flex sensor straight when the cell starts.
2. The notebook records a short baseline.
3. Bend the sensing area.
4. When the reading changes sufficiently from the baseline, the LED turns **ON**.
5. When the sensor returns close to its initial position, the LED turns **OFF**.

This is a basic functional test before introducing gradual brightness.

```
In [ ]: from gpiozero import MCP3008, LED
        from time import sleep

        flex = MCP3008(channel=0)
        led = LED(17)

        print("Keep the flex sensor straight...")
        sleep(2)

        samples = [flex.value for _ in range(20)]
        baseline = sum(samples) / len(samples)

        print(f"Baseline value: {baseline:.3f}")
```

```

print("Now bend and release the sensor. The test will run for 15 seconds.")

threshold = 0.05

for _ in range(150):
    reading = flex.value
    change = abs(reading - baseline)

    if change > threshold:
        led.on()
        state = "BENT → LED ON"
    else:
        led.off()
        state = "STRAIGHT → LED OFF"

    print(f"Reading: {reading:.3f} | Change: {change:.3f} | {state}", end="\n")
    sleep(0.1)

led.off()
led.close()
flex.close()
print("\nFlex-sensor ON/OFF test completed.")

```

12. From Binary Control to Gradual Control

The previous test treated the sensor in a simple binary way:

- approximately straight → LED OFF
- sufficiently bent → LED ON

Human descriptions are often more gradual. A bend might be described as:

- almost straight
- slightly bent
- moderately bent
- strongly bent

These descriptions are examples of **linguistic terms**.

Instead of forcing every sensor reading into only ON or OFF, the next experiment uses the amount of bending as a gradual value between 0 and 1 and maps that value to LED brightness.

13. Calibrate Straight and Bent Positions

Before controlling brightness, the notebook needs two reference values:

- the reading when the sensor is **straight**
- the reading when the sensor is **strongly but safely bent**

The code works whether the ADC value increases or decreases during bending.

```
In [ ]: from gpiozero import MCP3008
        from time import sleep

        flex = MCP3008(channel=0)

        def average_reading(sensor, samples=30, delay=0.03):
            values = []
            for _ in range(samples):
                values.append(sensor.value)
                sleep(delay)
            return sum(values) / len(values)

        print("Keep the flex sensor straight.")
        sleep(3)
        straight_value = average_reading(flex)
        print(f"Straight reference: {straight_value:.3f}")

        print("\nNow bend the flexible sensing area strongly but safely.")
        sleep(3)
        bent_value = average_reading(flex)
        print(f"Bent reference: {bent_value:.3f}")

        if abs(bent_value - straight_value) < 0.02:
            print("\nThe two reference values are very close. Check the wiring and r
        else:
            print("\nCalibration completed.")

        flex.close()
```

14. Fuzzy-Inspired Gradual LED Brightness

The next cell maps the calibrated bend amount continuously:

- **0.0** → approximately straight → LED off/very dim
- **0.5** → intermediate bend → medium brightness
- **1.0** → strongly bent → maximum brightness

It also calculates three overlapping linguistic memberships:

- **Straight**
- **Medium Bend**
- **Strong Bend**

A single bend value can partially belong to more than one linguistic term at the same time. This gradual overlap introduces the main intuition behind fuzzy linguistic variables.

```
In [ ]: from gpiozero import MCP3008, PWMLED
        from time import sleep
```

```

flex = MCP3008(channel=0)
led = PWMLED(17)

# Use the straight_value and bent_value obtained in the calibration cell.
difference = bent_value - straight_value

if abs(difference) < 0.02:
    raise ValueError("Calibration range is too small. Run the calibration cell")

def clamp(value, low=0.0, high=1.0):
    return max(low, min(high, value))

def bend_amount(reading):
    # Works whether the ADC reading rises or falls when the sensor bends.
    return clamp((reading - straight_value) / difference)

def memberships(x):
    straight = clamp(1 - 2*x)
    medium = clamp(1 - 2*abs(x - 0.5))
    strong = clamp(2*x - 1)
    return straight, medium, strong

print("Bend and release the flex sensor. The test will run for 20 seconds.")

for _ in range(200):
    reading = flex.value
    bend = bend_amount(reading)

    straight_m, medium_m, strong_m = memberships(bend)

    # Continuous physical output
    led.value = bend

    print(
        f"ADC: {reading:.3f} | Bend: {bend:.2f} | "
        f"Straight: {straight_m:.2f} | Medium: {medium_m:.2f} | "
        f"Strong: {strong_m:.2f} | LED brightness: {led.value:.2f}",
        end="\r"
    )

    sleep(0.1)

led.off()
led.close()
flex.close()
print("\nGradual flex-sensor experiment completed.")

```

15. What This Demonstrates

You have now moved through three levels of physical interaction:

1. **LED output:** the Raspberry Pi controls a physical device.
2. **Flex-sensor input:** the Raspberry Pi receives information about a physical action.
3. **Input-to-output mapping:** the amount of bending controls LED brightness.

The final experiment also introduces a key Computing with Words idea: a physical measurement can be interpreted through overlapping linguistic descriptions rather than only rigid ON/OFF categories.

This is an introductory demonstration. More complete fuzzy systems can later use membership functions, rule sets, inference, aggregation, and defuzzification.

16. Troubleshooting

LED does not turn on

- check the LED polarity
- check GPIO17 and GND
- confirm that the LED has a 220–330 Ω series resistor

Flex-sensor value does not change

- confirm that SPI is enabled
- check the MCP3008 orientation
- check 3.3 V and GND
- confirm that the voltage-divider midpoint is connected to MCP3008 CH0
- make sure the flex sensor is being bent along its sensing area, not at the terminals

MCP3008 cannot be accessed

- verify the SPI wiring
- verify that SPI was enabled through `raspi-config`
- reboot the Raspberry Pi after enabling SPI if required

LED brightness appears reversed

The calibration code is designed to handle increasing or decreasing ADC readings. Re-run the straight/bent calibration and make sure the straight and bent positions were held correctly.

Readings fluctuate slightly

Small changes are normal for analog measurements. The calibration cells average several samples to make the experiment more stable.

17. Safe Raspberry Pi Shutdown

Use this only when you are completely finished with the Raspberry Pi.

The shutdown command below is intentionally **commented out** so that it cannot run accidentally if several cells are executed together.

To shut down:

1. remove the `#` characters from the two lines below
2. run only this cell
3. wait until the Raspberry Pi has completely shut down before disconnecting power

Safe shutdown

When you finish working with the Raspberry Pi, shut it down safely before disconnecting the power cable.

This helps protect the microSD card and gives the system a cleaner start the next time you use it.

```
In [ ]: # import os  
        # os.system("sudo shutdown -h now")
```

18. Summary

In this tutorial, you connected and tested an LED with Raspberry Pi GPIO, extended the breadboard circuit with a flex sensor and MCP3008 ADC, and observed how physical bending can be converted into a digital reading.

You then used the bend amount first as a simple ON/OFF condition and finally as a gradual value controlling LED brightness. The overlapping linguistic descriptions **Straight**, **Medium Bend**, and **Strong Bend** provide an introductory bridge from physical sensing to Computing with Words and fuzzy reasoning.